

Paimon Hive Catalog

核心架构设计文档

核心架构设计 · 设计模式 · 流程图 · 时序图 · 调用链

模块: paimon-hive-catalog

分支: release-1.3.1

生成工具: AI Architecture Analyzer

生成日期: 2026年03月28日

第1章 模块概述

1.1 模块定位

paimon-hive-catalog 是 Apache Paimon 数据湖表格式项目中的核心集成模块，负责将 Paimon 表管理能力与 Apache Hive Metastore (HMS) 进行深度整合。该模块实现了基于 Hive Metastore 的 Catalog 服务，使 Paimon 表的元数据可以通过 HMS 进行统一管理，从而让 Hive、Spark、Flink、Presto、Trino 等大数据引擎能够透明地访问 Paimon 表。

1.2 核心目标

- HMS 元数据统一管理 — 将 Paimon 表的数据库、表、分区等元数据同步到 Hive Metastore，实现与 Hive 生态的无缝对接
- 多引擎兼容 — 支持 Hive/Spark/Flink/Presto/Trino 等引擎通过 HMS 发现和访问 Paimon 表
- 分布式锁支持 — 基于 HMS lock API 提供表级排他锁，保证并发元数据操作的一致性
- Hive 表迁移与克隆 — 支持将原生 Hive 表迁移（migrate）或克隆（clone）为 Paimon 表
- Format 表支持 — 支持 Parquet/ORC/CSV/JSON/TEXT 等格式的外部 Hive 表作为 Format Table 访问
- Iceberg 兼容元数据 — 支持将 Paimon 表的 Iceberg 兼容元数据提交到 HMS，使 Iceberg Hive Catalog 也能访问

1.3 源码文件一览

文件名	包/子模块	职责描述
HiveCatalog.java	hive	核心类，继承 AbstractCatalog，实现完整的 HMS 元数据 CRUD 操作
HiveCatalogFactory.java	hive	CatalogFactory SPI 实现，标识符为 hive
HiveCatalogLoader.java	hive	可序列化的 CatalogLoader，支持分布式环境加载
HiveCatalogLock.java	hive	基于 HMS lock API 的表级排他锁实现
HiveCatalogLockContext.java	hive	锁上下文，封装 HiveConf 和客户端类名
HiveCatalogLockFactory.java	hive	CatalogLockFactory SPI 实现
HiveCatalogOptions.java	hive	配置选项常量类（hive-conf-dir 等）
HiveTableUtils.java	hive	Hive 表转 Paimon Format Table Schema 工具
HiveAlterTableUtils.java	hive	兼容多版本 Hive 的 alter_table 工具
LocationHelper.java	hive	位置管理策略接口
StorageLocationHelper.java	hive	使用 StorageDescriptor 的标准位置策略

文件名	包/子模块	职责描述
TBPropertiesLocationHelper.java	hive	使用 table properties 的替代位置策略
RetryingMetaStoreClientFactory.java	hive	多版本兼容的 HMS 重试客户端工厂
SerializableHiveConf.java	hive	HiveConf 可序列化包装器
HiveCloneExtractor.java	hive.clone	克隆提取器接口，SPI 扩展点
HiveCloneUtils.java	hive.clone	克隆工具类（schema 转换、文件列表、格式解析）
HivePartitionFiles.java	hive.clone	分区文件信息 POJO
HiveTableCloneExtractor.java	hive.clone	默认 Hive 表克隆提取器实现
HiveMigrator.java	hive.migrate	Hive 表迁移到 Paimon 表，支持并行
CachedClientPool.java	hive.pool	基于 Caffeine 缓存的 HMS 连接池
HiveClientPool.java	hive.pool	HMS 客户端连接池实现
IcebergHiveMetadataCommitter.java	iceberg	Iceberg 元数据提交到 HMS
IcebergHiveMetadataCommitterFactory.java	iceberg	Iceberg 元数据提交器工厂

第2章 核心架构设计

2.1 整体分层架构

paimon-hive-catalog 模块采用四层架构设计：SPI 工厂层、Catalog 核心层、连接池层、工具/扩展层。整体架构围绕 HiveCatalog 核心类展开，通过 SPI 机制实现工厂发现，通过策略模式处理位置管理，通过连接池管理 HMS 客户端连接。

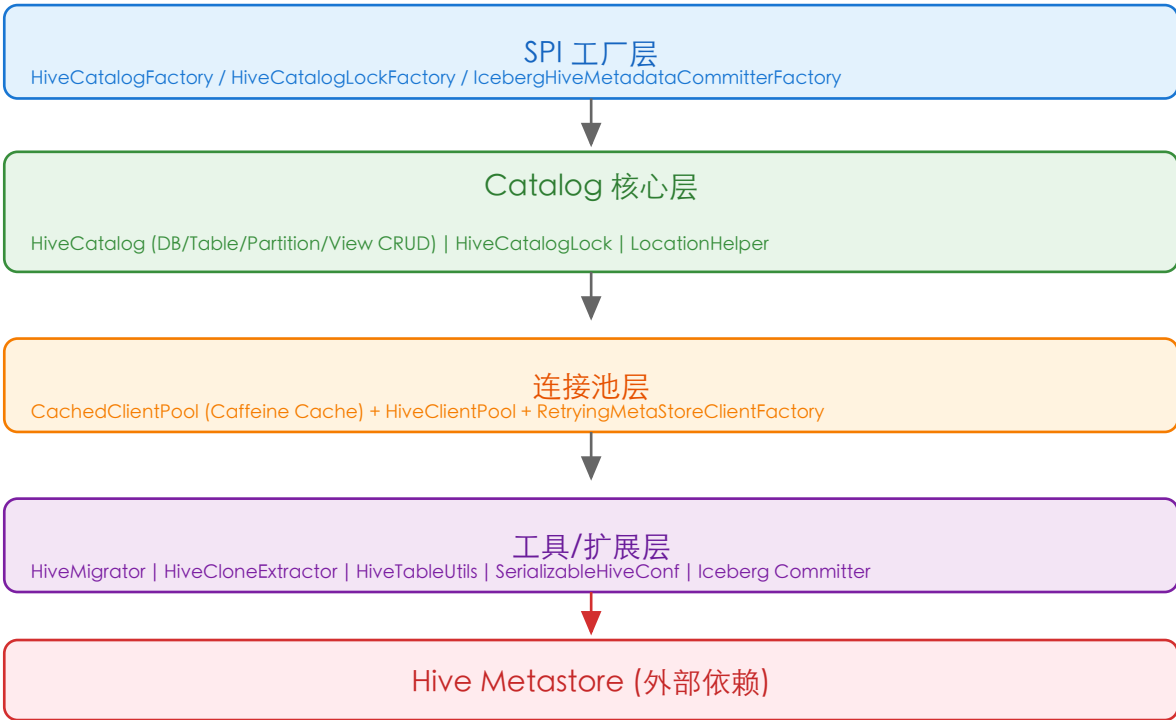


图2-1 paimon-hive-catalog 整体分层架构图

2.2 类继承与接口关系

模块中的核心类通过继承和实现 Paimon 核心模块定义的抽象类和接口来实现功能，形成清晰的依赖反转架构。

本模块类	父类/接口	关系	所属模块
HiveCatalog	AbstractCatalog	继承	paimon-core
HiveCatalogFactory	CatalogFactory	实现	paimon-core
HiveCatalogLoader	CatalogLoader	实现	paimon-core
HiveCatalogLock	CatalogLock	实现	paimon-core
HiveCatalogLockContext	CatalogLockContext	实现	paimon-core
HiveCatalogLockFactory	CatalogLockFactory	实现	paimon-core

本模块类	父类/接口	关系	所属模块
CachedClientPool	ClientPool	实现	paimon-core
HiveClientPool	ClientPool.ClientPoolImpl	继承	paimon-core
HiveMigrator	Migrator	实现	paimon-core
IcebergHiveMetadataCommitter	IcebergMetadataCommitter	实现	paimon-core
StorageLocationHelper	LocationHelper	实现	本模块
TBPropertiesLocationHelper	LocationHelper	实现	本模块

第3章 核心类设计模式详解

3.1 HiveCatalog — Facade（门面）模式 + Template Method（模板方法）模式

HiveCatalog 是整个模块的核心入口类（1875行），继承自 AbstractCatalog，实现了基于 Hive Metastore 的完整 Catalog 操作。它作为 Facade，将 HMS 客户端交互、位置管理、锁管理、分区同步等复杂逻辑统一封装在一个简洁的接口背后。同时，它通过覆写 AbstractCatalog 定义的模板方法（如 createDatabaseImpl、createTableImpl、dropTableImpl 等）实现 Template Method 模式。

关键方法：

- createTableImpl() — 创建 Paimon 表：先校验 HMS 中不存在同名表，然后在文件系统创建 schema，最后在 HMS 创建 Hive 表对象
- dropTableImpl() — 删除表：区分外部表和托管表，外部表仅删元数据，托管表同时删文件
- alterTableImpl() — 修改表：先提交 schema 变更到文件系统，然后同步到 HMS
- repairTable() — 修复表：重新同步文件系统上的 schema 到 HMS，并修复分区信息
- createHiveConf() — 静态工具方法，创建 HiveConf，支持 hive-conf-dir / hadoop-conf-dir / classpath / env 多种配置来源

3.2 LocationHelper — Strategy（策略）模式

LocationHelper 定义了位置管理的策略接口，有两个实现类。在 HiveCatalog 构造函数中，根据 location-in-properties 配置动态选择策略实例，体现了经典的 Strategy 设计模式。

- StorageLocationHelper — 标准策略：使用 Hive 原生的 StorageDescriptor.location 和 Database.locationUri 存储路径，路径创建/删除由 HMS 管理
- TBPropertiesLocationHelper — 替代策略：将路径存储在 table/database properties 中，避免 HMS 直接访问文件系统。适用于 S3/OSS 等对象存储场景，需手动通过 FileIO 管理路径

3.3 CachedClientPool — Object Pool（对象池）模式 + Proxy（代理）模式

CachedClientPool 实现了 ClientPool 接口，使用 Caffeine Cache 管理 HiveClientPool 实例。它既是 Object Pool 模式（复用 HMS 客户端连接），又是 Proxy 模式（将 run/execute 请求代理到缓存的 HiveClientPool）。缓存键支持 UGI/user_name/conf 多维度组合，实现多租户隔离。

- extractKey() — 根据配置的 cache keys 生成缓存键，支持 UGI、user_name 和任意配置项组合
- clientPool() — 从 Caffeine 缓存获取或创建 HiveClientPool 实例
- init() — 懒初始化静态缓存实例，配置过期策略和定时清理线程

3.4 HiveCatalogLock — 分布式锁模式 + Exponential Backoff

HiveCatalogLock 实现了 CatalogLock 接口，利用 HMS 的 lock/unlock/checkLock API 实现表级排他锁。锁等待采用 指数退避 (Exponential Backoff) 策略，每次重试睡眠时间翻倍，上限为 checkMaxSleep 配置值。

- lock() — 请求 EXCLUSIVE 表锁，循环 checkLock 等待获取，超时则释放并抛异常
- unlock() — 释放锁，失败仅记录警告不抛异常

3.5 RetryingMetaStoreClientFactory — Abstract Factory (抽象工厂) + Adapter (适配器) 模式

RetryingMetaStoreClientFactory 通过反射适配 Hive 1.x/2.x/3.x 不同版本的 RetryingMetaStoreClient.getProxy() 方法签名，实现了跨版本兼容的 HMS 客户端创建。它预定义了多种参数签名的 HiveMetastoreProxySupplier，按顺序尝试匹配当前 Hive 版本，体现了 Abstract Factory 和 Adapter 模式的组合。

3.6 SerializableHiveConf — Decorator (装饰器) 模式

SerializableHiveConf 将不可序列化的 HiveConf 包装为可序列化的类。使用懒序列化机制：只在真正需要序列化时才将 HiveConf 写入字节数组，反序列化时再恢复。这种 Decorator 模式为 HiveConf 添加了序列化能力而不改变其原有接口。

3.7 HiveCloneExtractor — SPI (服务提供者接口) + Chain of Responsibility (责任链) 模式

HiveCloneExtractor 是克隆提取器接口，通过 FactoryUtil.discoverFactories() 使用 SPI 机制发现所有实现类。getExtractor() 方法遍历所有注册的提取器，用 matches() 方法匹配表类型，匹配失败则回退到默认的 HiveTableCloneExtractor。这是 SPI 扩展与责任链模式的结合。

第4章 核心流程图

4.1 创建表流程 (createTableImpl)

创建 Paimon 表是 HiveCatalog 最核心的操作之一，涉及 HMS 检查、文件系统 Schema 创建和 HMS 表注册三个阶段。

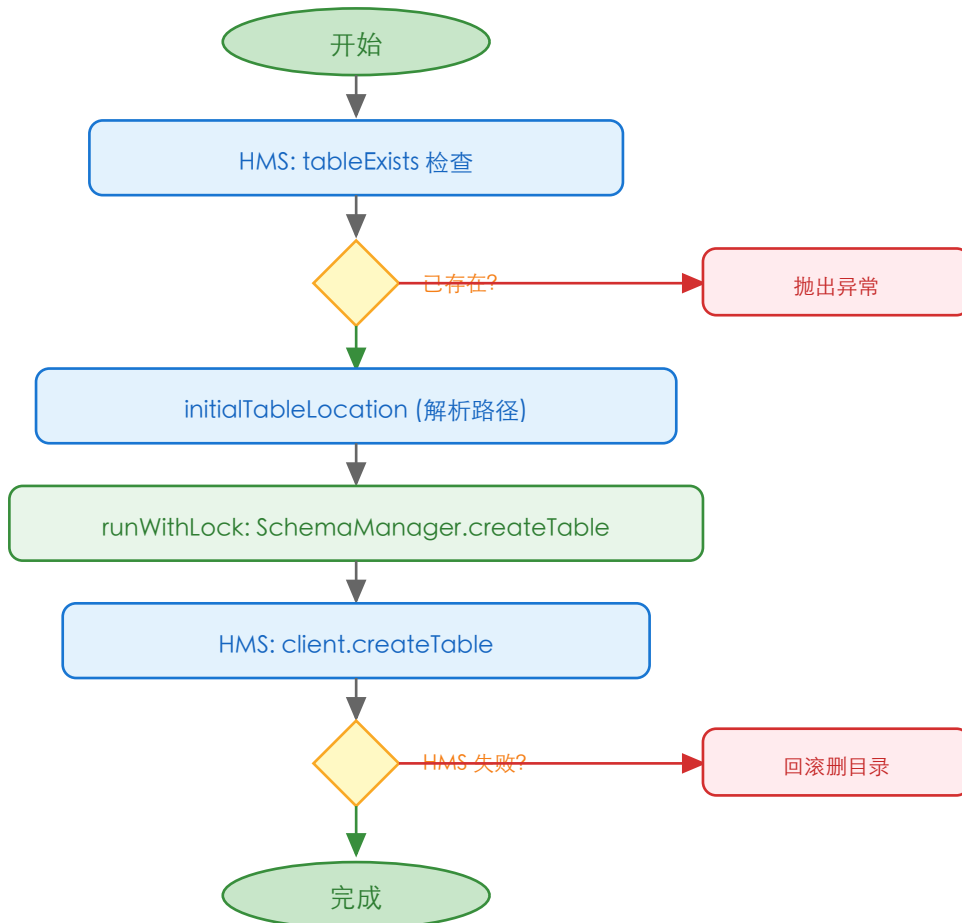


图4-1 createTableImpl 创建表流程图

4.2 Hive 表迁移流程 (HiveMigrator.executeMigrate)

HiveMigrator 将 Hive 表迁移为 Paimon 表，支持分区表的并行迁移和失败回滚。

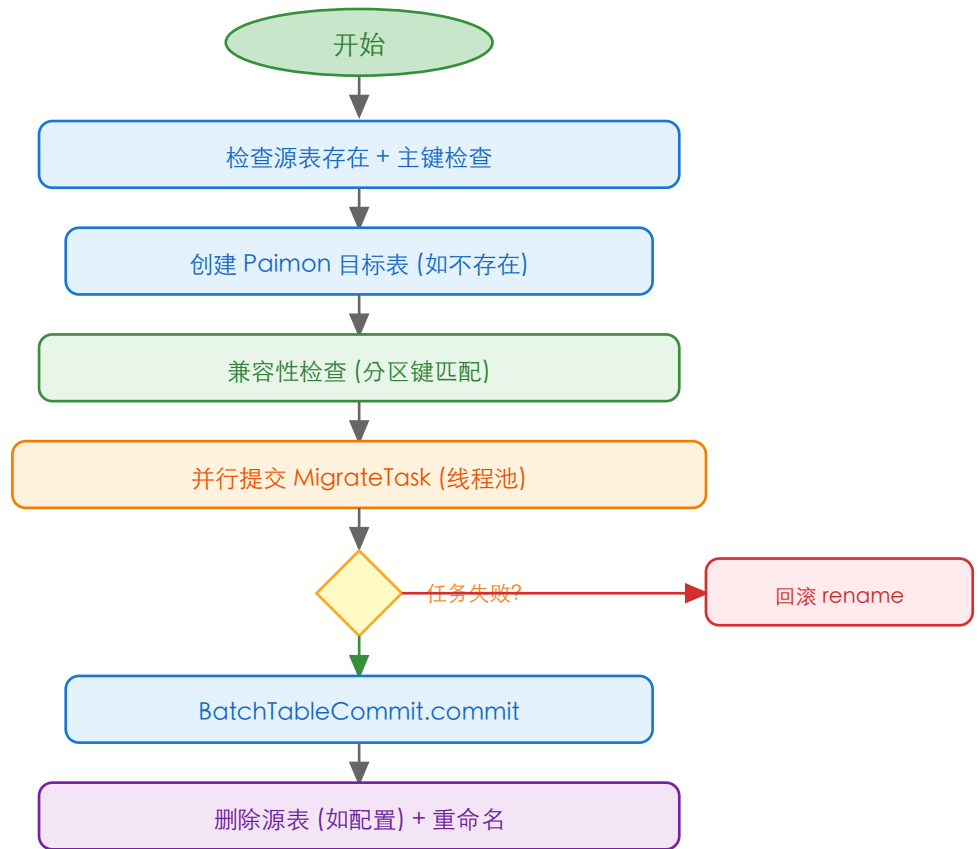


图4-2 HiveMigrator 迁移流程图

4.3 HMS 客户端连接获取流程

HiveCatalog 通过双重检查锁（DCL）模式懒初始化 CachedClientPool，再从 Caffeine 缓存获取或创建 HiveClientPool。

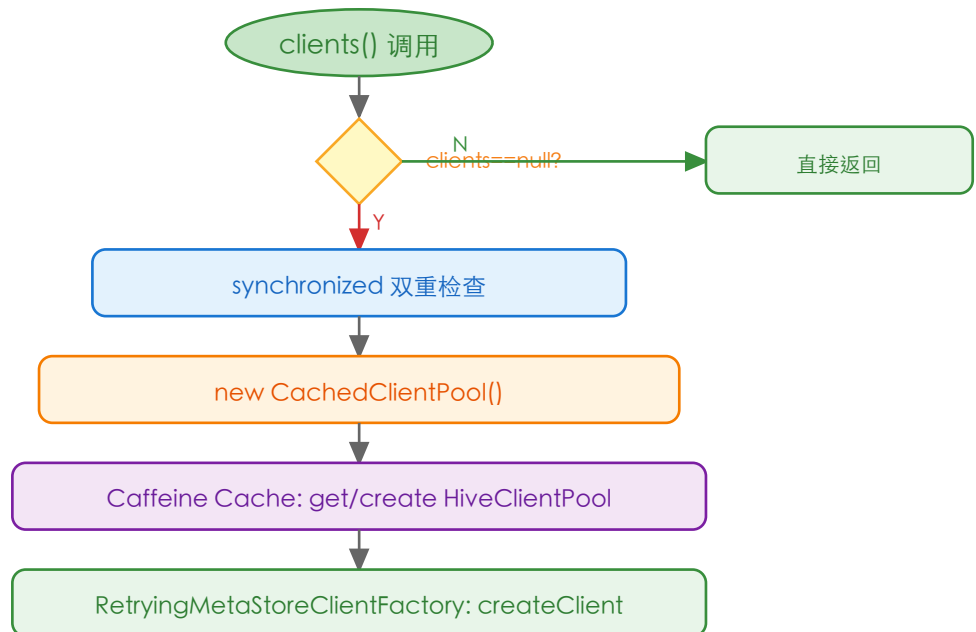


图4-3 HMS 客户端连接获取流程

第5章 核心时序图

5.1 创建表交互时序图

展示用户通过 HiveCatalog 创建 Paimon 表时，各组件之间的交互序列。

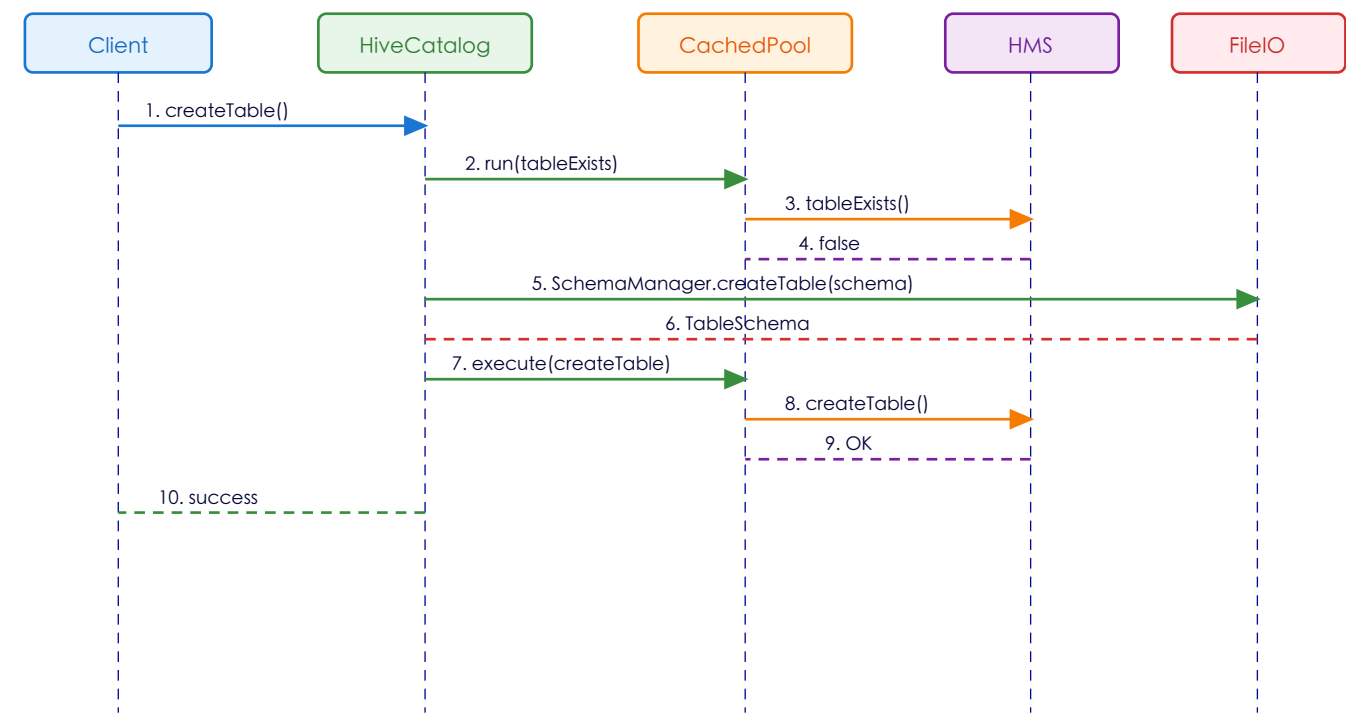


图5-1 创建表交互时序图

5.2 分布式锁获取与释放时序图

展示 HiveCatalogLock 在 runWithLock 场景下的锁获取、业务执行和锁释放交互。

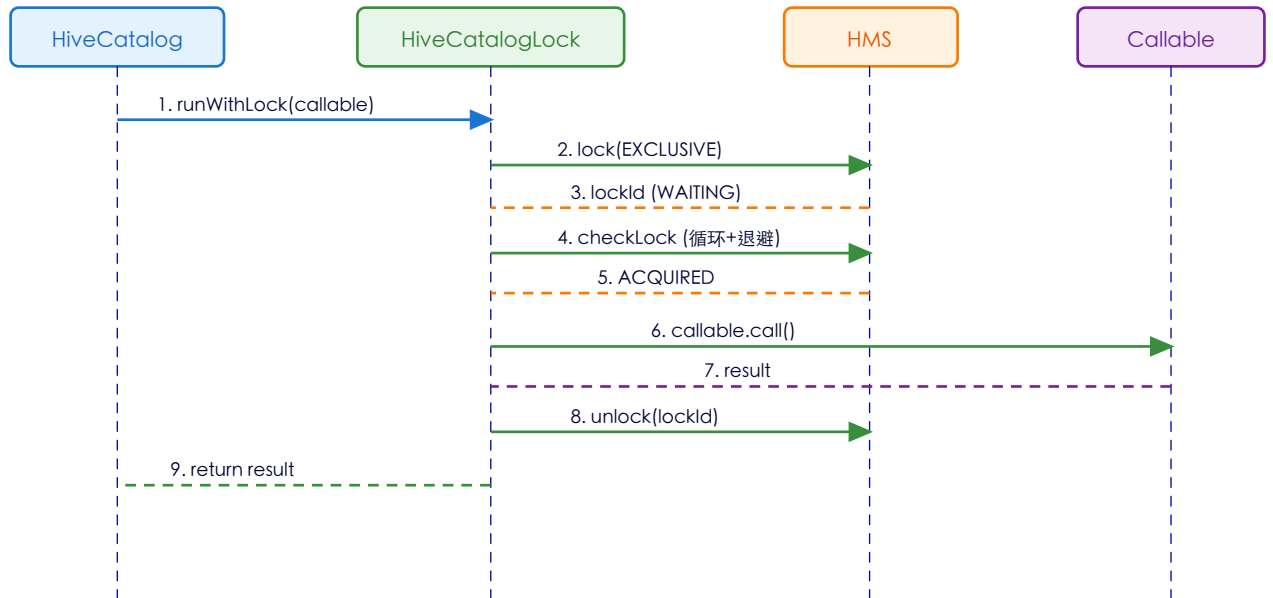


图5-2 分布式锁获取与释放时序图

5.3 Iceberg 元数据提交时序图

展示 IcebergHiveMetadataCommitter 将 Paimon 表的 Iceberg 兼容元数据提交到 HMS 的过程。

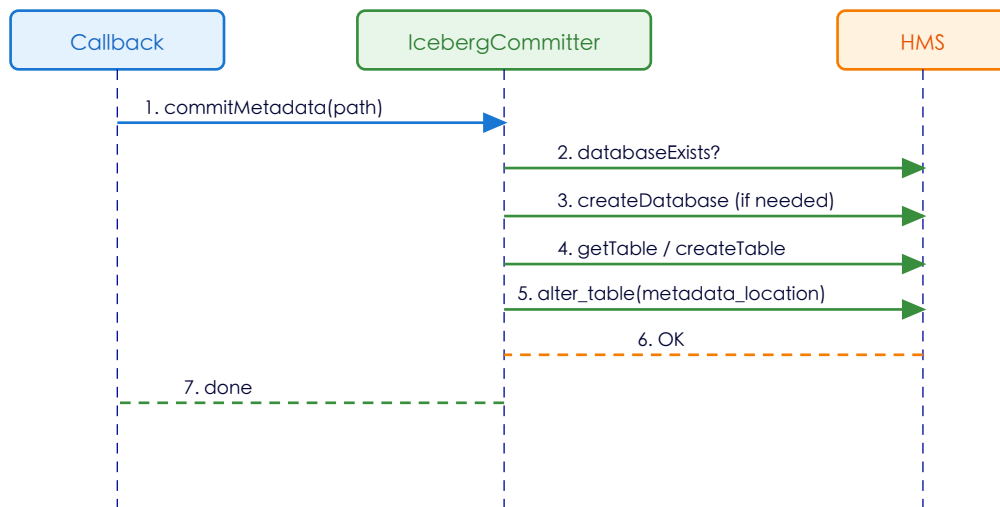


图5-3 Iceberg 元数据提交时序图

第6章 关键场景调用链

6.1 场景一: 创建 Hive Catalog 实例

触发条件：引擎（Flink/Spark）通过 SPI 加载 HiveCatalogFactory 创建 Catalog 实例

步骤	类.方法	说明
1	HiveCatalogFactory.create(context)	SPI 入口，接收 CatalogContext
2	HiveCatalog.createHiveCatalog(context)	静态工厂方法
3	HiveCatalog.createHiveConf(context)	加载 hive-conf-dir / hadoop-conf-dir / URI 配置创建 HiveConf
4	FileIO.get(uri, context)	获取文件系统 IO 实例
5	new HiveCatalog(fileIO, hiveConf, ...)	构造函数，选择 LocationHelper 策略

6.2 场景二: 创建 Paimon 表

触发条件：用户执行 CREATE TABLE 语句

步骤	类.方法	说明
1	HiveCatalog.createTableImpl(identifier, schema)	覆写的模板方法入口
2	clients().run(client.tableExists())	检查 HMS 中是否已有同名表
3	initialTableLocation(options, identifier)	解析表路径（外部表/托管表）
4	runWithLock(id, schemaManager.createTable)	获取分布式锁后在文件系统创建 schema
5	createHiveTable(identifier, schema, location)	构建 HMS Table 对象（InputFormat/OutputFormat/SerDe）
6	clients().execute(client.createTable(table))	在 HMS 中注册表

6.3 场景三: 修改表结构 (ALTER TABLE)

触发条件：用户执行 ALTER TABLE 修改列或选项

步骤	类.方法	说明
1	HiveCatalog.alterTableImpl(id, changes)	覆写的模板方法入口

步骤	类.方法	说明
2	getHmsTable(identifier)	从 HMS 获取当前表信息
3	runWithLock(id, schemaManager.commitChanges)	加锁提交 schema 变更到文件系统
4	alterTableToHms(table, id, newSchema, removed)	同步变更到 HMS
5	HiveAlterTableUtils.alterTable(client, id, table)	兼容多版本 Hive 的 alter_table 调用

6.4 场景四: 克隆 Hive 表到 Paimon

触发条件：用户执行 CLONE TABLE 操作

步骤	类.方法	说明
1	HiveCloneUtils.hiveTableToPaimonSchema()	获取 HMS 表信息，检查主键，提取 Schema
2	HiveCloneExtractor.getExtractor(table)	SPI 匹配合适的克隆提取器
3	extractor.extractSchema(client, table, ...)	提取字段定义
4	extractor.extractPartitionKeys(table)	提取分区键
5	extractor.extractOptions(table)	提取选项（格式、压缩等）
6	HiveCloneUtils.listFiles()	列出源表所有文件（分区/非分区）

6.5 场景五: 修复表 (repairTable)

触发条件：用户执行 MSCK REPAIR TABLE 或 Paimon repair 命令

步骤	类.方法	说明
1	HiveCatalog.repairTable(identifier)	入口
2	tableSchemaInFileSystem(location, branch)	从文件系统读取最新 schema
3	getHmsTable(identifier)	尝试从 HMS 获取当前表
4	createHiveTable(id, schema, location)	构建新的 HMS Table 对象
5	alterTableToHms / createTable	更新或创建 HMS 表
6	createPartitions(identifier, partitions)	修复分区信息，同步到 HMS

6.6 场景六: Iceberg 兼容元数据提交

触发条件：Paimon 表配置了 Iceberg 兼容模式，提交 snapshot 后回调

步骤	类.方法	说明
1	IcebergHiveMetadataCommitter.commitMetadata()	回调入口
2	databaseExists() / createDatabase()	确保 Iceberg 数据库在 HMS 中存在
3	tableExists() / createTable()	确保 Iceberg 表在 HMS 中存在
4	设置 metadata_location / previous_metadata_location	更新表属性中的元数据路径
5	alter_table_with_environmentContext()	提交变更到 HMS，支持 CASCADE 和跳过统计更新

第7章 配置与扩展

7.1 配置属性

配置项	默认值	说明
hive-conf-dir	(无)	hive-site.xml 所在目录路径
hadoop-conf-dir	(无)	core-site.xml 等 Hadoop 配置目录
metastore.client.class	HiveMetaStoreClient	HMS 客户端实现类
location-in-properties	false	是否将位置存储在 table properties 中 (S3/OSS 场景)
client-pool-cache.eviction-interval-ms	300000 (5min)	客户端池缓存过期间隔
client-pool-cache.keys	(无)	缓存键维度: ugi / user_name / conf:xxx

7.2 SPI 扩展点

模块通过 META-INF/services/org.apache.paimon.factories.Factory 注册了三个 SPI 实现：

- HiveCatalogFactory — CatalogFactory 扩展，标识符为 "hive"
- HiveCatalogLockFactory — CatalogLockFactory 扩展，标识符为 "hive"
- IcebergHiveMetadataCommitterFactory — IcebergMetadataCommitterFactory 扩展

7.3 克隆提取器扩展

HiveCloneExtractor 接口支持 SPI 扩展。用户可以实现自定义的克隆提取器（如支持 Iceberg/Delta Lake 格式的 Hive 表），通过 FactoryUtil.discoverFactories() 自动发现。默认回退到 HiveTableCloneExtractor。

第8章 设计亮点与总结

8.1 设计亮点

- 多版本 Hive 兼容：RetryingMetaStoreClientFactory 通过反射适配 Hive 1.x/2.x/3.x 的不同 API 签名，HiveAlterTableUtils 兼容有无 EnvironmentContext 的 alter_table 方法，确保在各版本 Hive 环境中均可正常运行
- 策略模式解耦位置管理：LocationHelper 接口将位置存储逻辑与 Catalog 核心操作解耦，通过 location-in-properties 配置切换标准模式与对象存储模式，无需修改核心代码
- 多租户连接池隔离：CachedClientPool 支持 UGI/user_name/conf 多维度缓存键，实现不同用户和配置间的 HMS 连接池隔离
- 完善的失败回滚机制：createTableImpl 在 HMS 注册失败时回滚文件系统，HiveMigrator 在任务失败时回滚所有 rename 操作
- 双向元数据同步：repair 机制支持从文件系统到 HMS 的元数据修复（repairTable），Iceberg 兼容模块支持从 Paimon 到 HMS 的 Iceberg 元数据同步
- SPI 扩展性：CatalogFactory、CatalogLockFactory、IcebergMetadataCommitterFactory、HiveCloneExtractor 均通过 SPI 机制注册，支持无侵入式扩展

8.2 设计限制

- HMS 单点依赖：所有元数据操作依赖 Hive Metastore 服务的可用性，HMS 故障会导致 Catalog 不可用
- 分区同步延迟：分区信息的 HMS 同步依赖 repair 或写入触发，可能出现 HMS 分区信息滞后
- 锁粒度限制：HiveCatalogLock 只支持表级排他锁，无法实现更细粒度（分区级/行级）的并发控制
- 迁移限制：HiveMigrator 仅支持 Append Only 且 Bucket Unaware 的目标表，不支持主键表迁移

8.3 质量评估

评估维度	评分	说明
代码结构	★★★★☆	分层清晰，核心类 HiveCatalog 较大（1875行）但职责明确
设计模式运用	★★★★★	策略、工厂、对象池、门面、装饰器、SPI 等模式运用恰当
可扩展性	★★★★☆	SPI 扩展点丰富，LocationHelper 策略模式灵活
版本兼容性	★★★★★	Hive 1.x/2.x/3.x 全面兼容，shaded 包场景也有覆盖
错误处理	★★★★☆	创建表有回滚机制，迁移有并行回滚，锁有超时保护
可测试性	★★★★☆	测试覆盖连接池缓存键、序列化、clone 过滤等核心场景